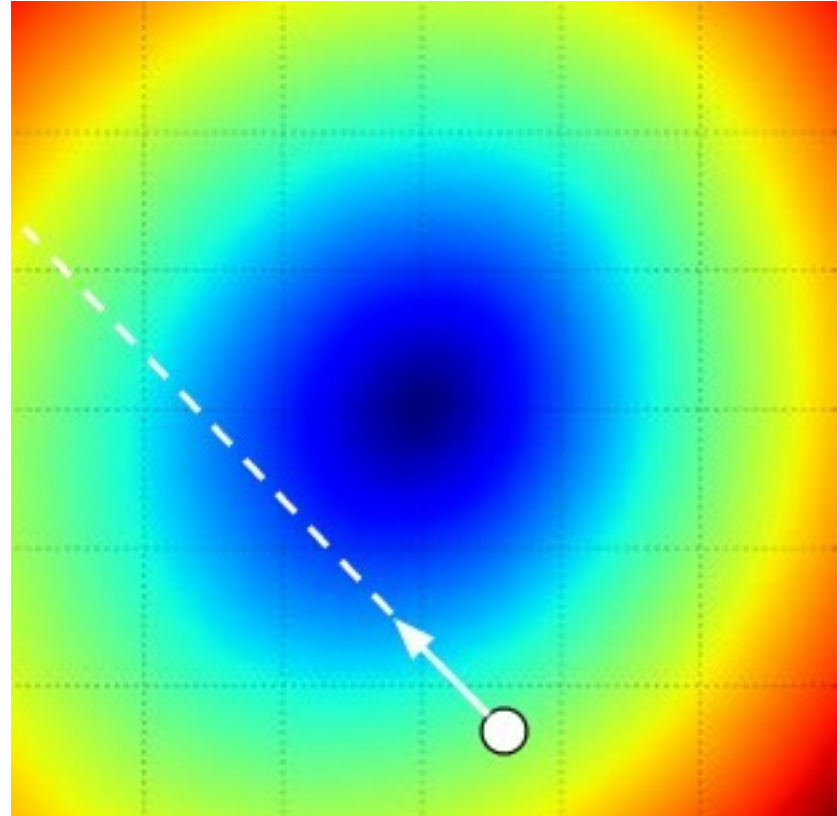


Optimization with gradient decent

- Update W by following the gradient of the loss function with respect to W

$$w_t = w_{t-1} - \alpha \frac{\partial \mathcal{L}}{\partial w}$$



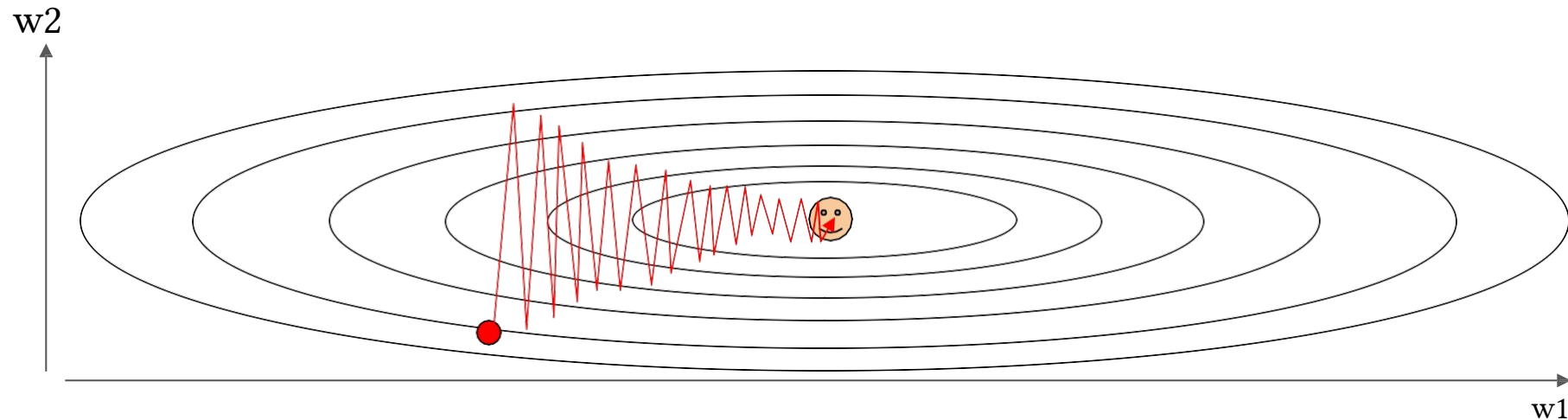
Stochastic Gradient Descent (SGD)

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

- Approximate sum using a minibatch
- Pros:
 - **Efficiency**: Scales well to large datasets and streaming data
 - **Regularization**: Noise in updates helps avoiding local minima
- Hyperparameters:
 - Batch size
 - Data sampling

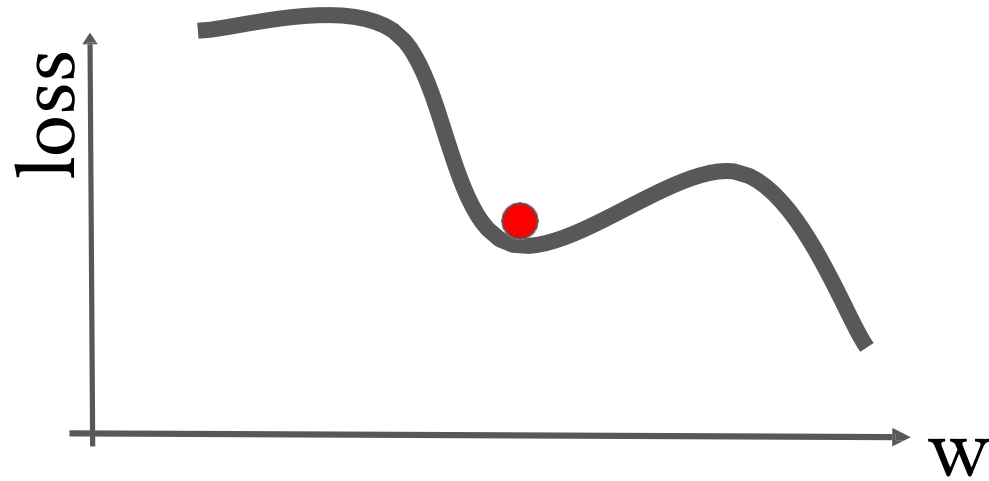
Problem #1

- Very slow progress along shallow dimension, jitter along steep direction

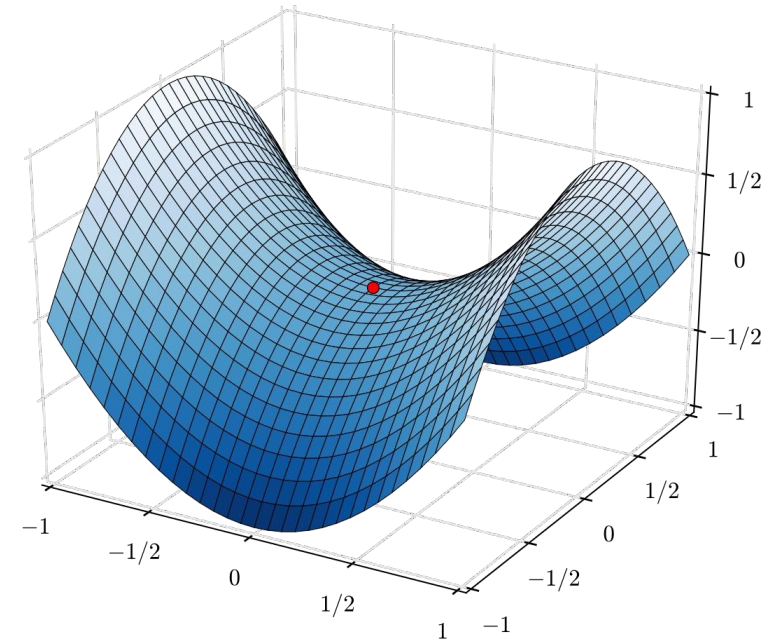


Problem #2

- Local minima or saddle point; Zero gradient



Local minimum



Saddle point

SGD + Momentum

- Continue direction as the previous iterations
- Build up “velocity” as a running mean of gradients
- Rho gives “momentum”; typically, rho=0.9 or 0.99

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
for t in range(num_steps):  
    dw = compute_gradient(w)  
    w -= learning_rate * dw
```

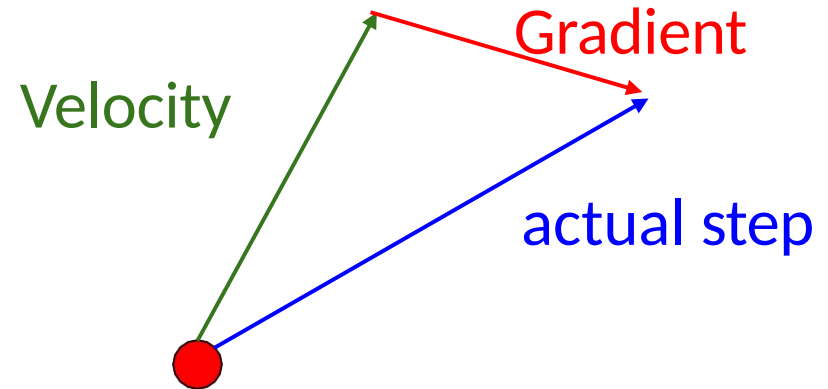
$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
v = 0  
for t in range(num_steps):  
    dw = compute_gradient(w)  
    v = rho * v + dw  
    w -= learning_rate * v
```

Nesterov Momentum

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t + \rho v_t)$$
$$x_{t+1} = x_t + v_{t+1}$$



“Look ahead” to the point where updating using velocity would take us; compute gradient there and mix it with velocity to get actual update direction

AdaGrad (adaptive learning rates)

- Added element-wise scaling of the gradient based on the historical sum of squares in each dimension

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared += dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

RMSProp: “Leak AdaGrad”

- Prevents the learning rate from shrinking too much

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared += dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

AdaGrad

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

RMSProp

Adam (almost): RMSProp + Momentum

```
moment1 = 0
moment2 = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    moment1 = beta1 * moment1 + (1 - beta1) * dw
    moment2 = beta2 * moment2 + (1 - beta2) * dw * dw
    w -= learning_rate * moment1 / (moment2.sqrt() + 1e-7)
```

Adam

Momentum

AdaGrad / RMSProp

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

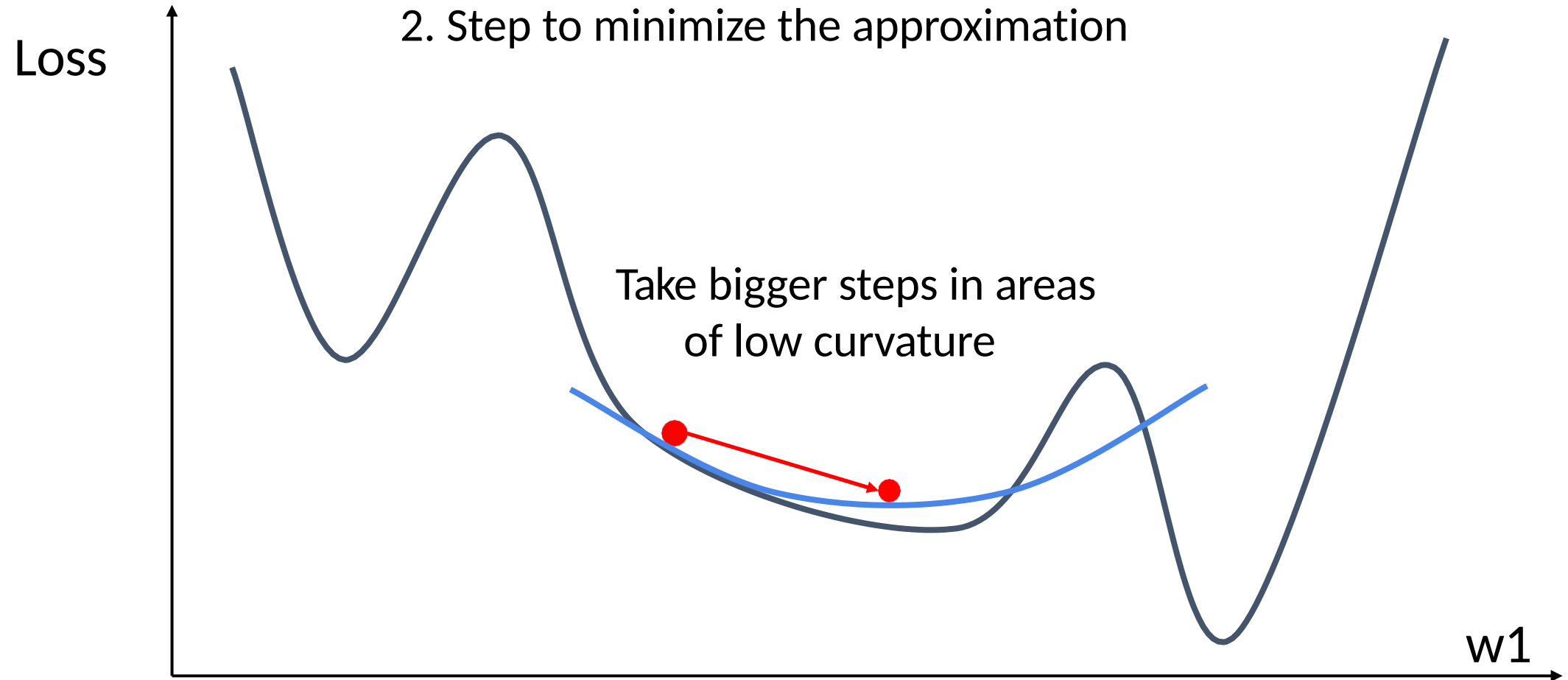
RMSProp

Optimization Algorithm Comparison

Algorithm	Tracks first moments (Momentum)	Tracks second moments (Adaptive learning rates)	Leaky second moments
SGD	X	X	X
SGD+Momentum	✓	X	X
Nesterov	✓	X	X
AdaGrad	X	✓	X
RMSProp	X	✓	✓
Adam	✓	✓	✓

Second-Order Optimization

1. Use gradient and Hessian to make quadratic approximation
2. Step to minimize the approximation



Second-Order Optimization

Second-Order Taylor Expansion:

$$L(w) \approx L(w_0) + (w - w_0)^\top \nabla_w L(w_0) + \frac{1}{2} (w - w_0)^\top \mathbf{H}_w L(w_0) (w - w_0)$$

Solving for the critical point we obtain the Newton parameter update:

$$w^* = w_0 - \mathbf{H}_w L(w_0)^{-1} \nabla_w L(w_0)$$

Q: Why is this impractical?

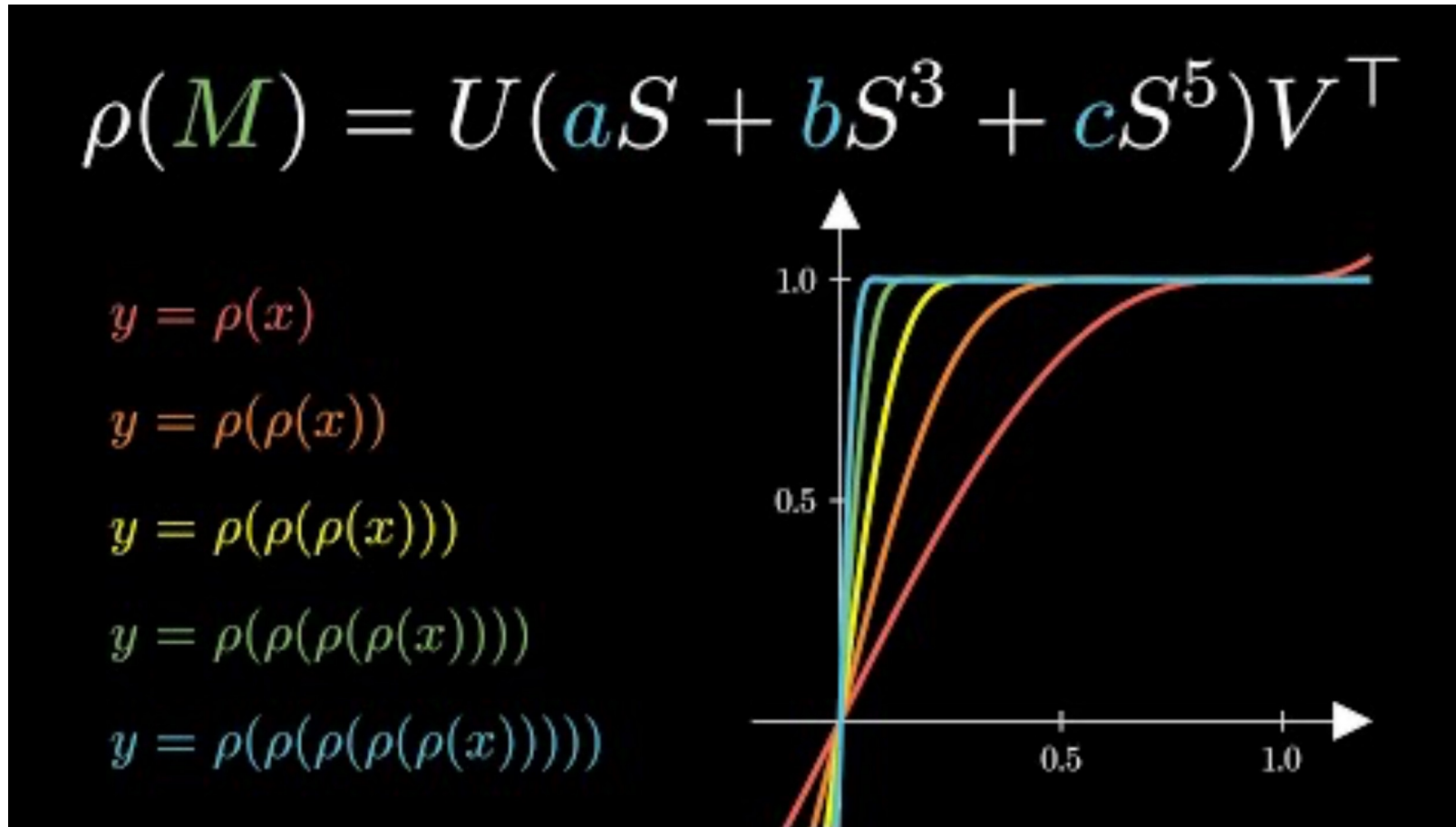
Hessian has $O(N^2)$ elements

Inverting takes $O(N^3)$

N = (Tens or Hundreds of)

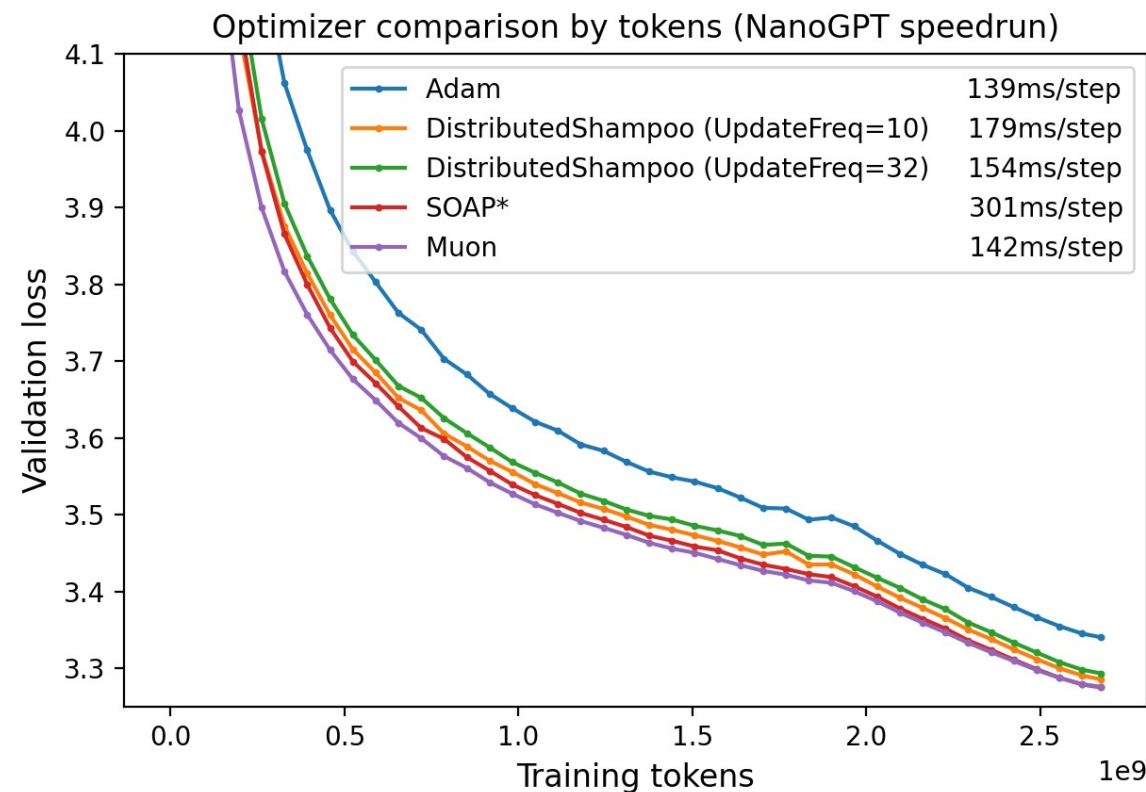
Millions

More recent methods (Muon)



More recent methods (Muon)

- `moment_1` and `moment_2` require two extra variables per parameter
- Linear functions, lead to sparse momentums



*SOAP is under active development. Future versions will significantly improve the wallclock overhead.